Develop Generative AI Apps in Azure

Build, optimize, and govern AI applications with Microsoft Foundry

Rick Beerendonk
rick@oblicum.com

glasspaper oblicum

© Rick Beerendonk

What You'll Learn

- Explain how Foundry, a project, and a model deployment fit together
- Send a first request and continue a conversation over many turns
- Let a model call tools instead of only producing text
- Choose between prompting, retrieval, and fine-tuning
- Keep an AI application safe, observable, and inside its limits

© Rick Beerendonk

Problem: A Model Alone Is Not an Application

A language model is a function: text in, text out. That is all it does.

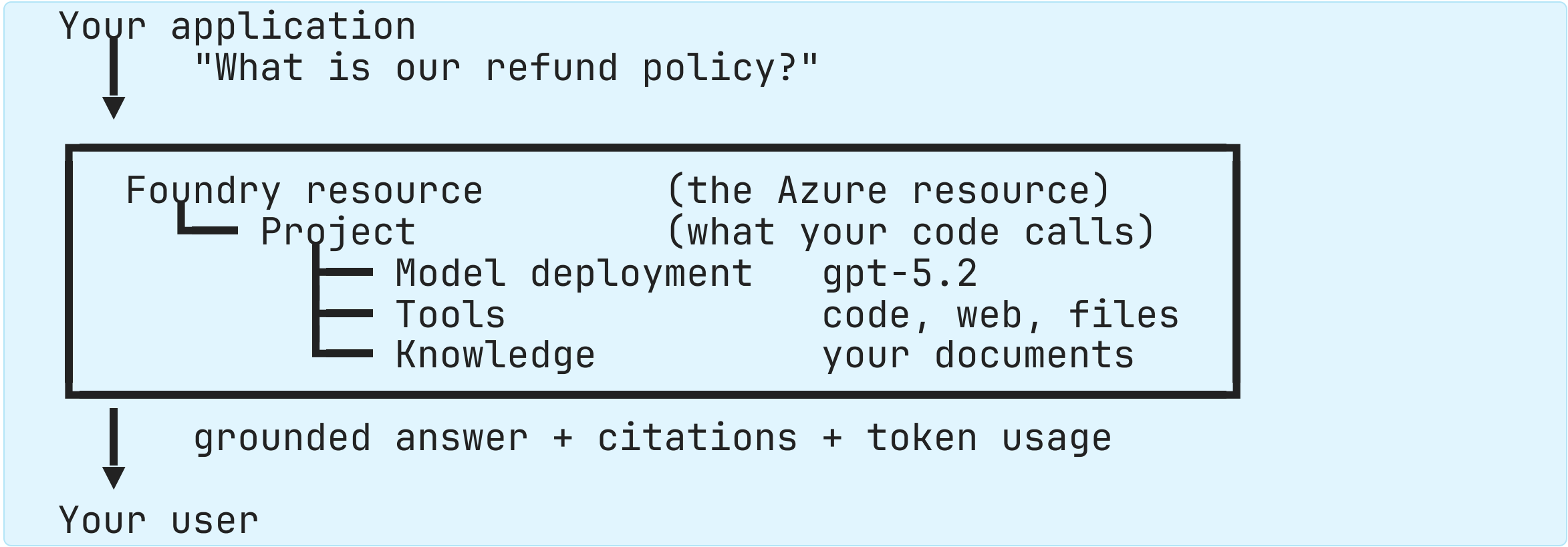
- ❌ It has never seen your data
- ❌ It forgets everything after each call
- ❌ It cannot look anything up or take an action
- ❌ It has no identity, no budget, no audit trail

Need: a place to host models, connect them to your data and tools, and watch what they do.

That place is **Microsoft Foundry**. Everything in this presentation is one of those missing pieces.

© Rick Beerendonk

The Big Picture



One resource holds projects. A project holds everything one application needs.

© Rick Beerendonk

Decoding the Jargon

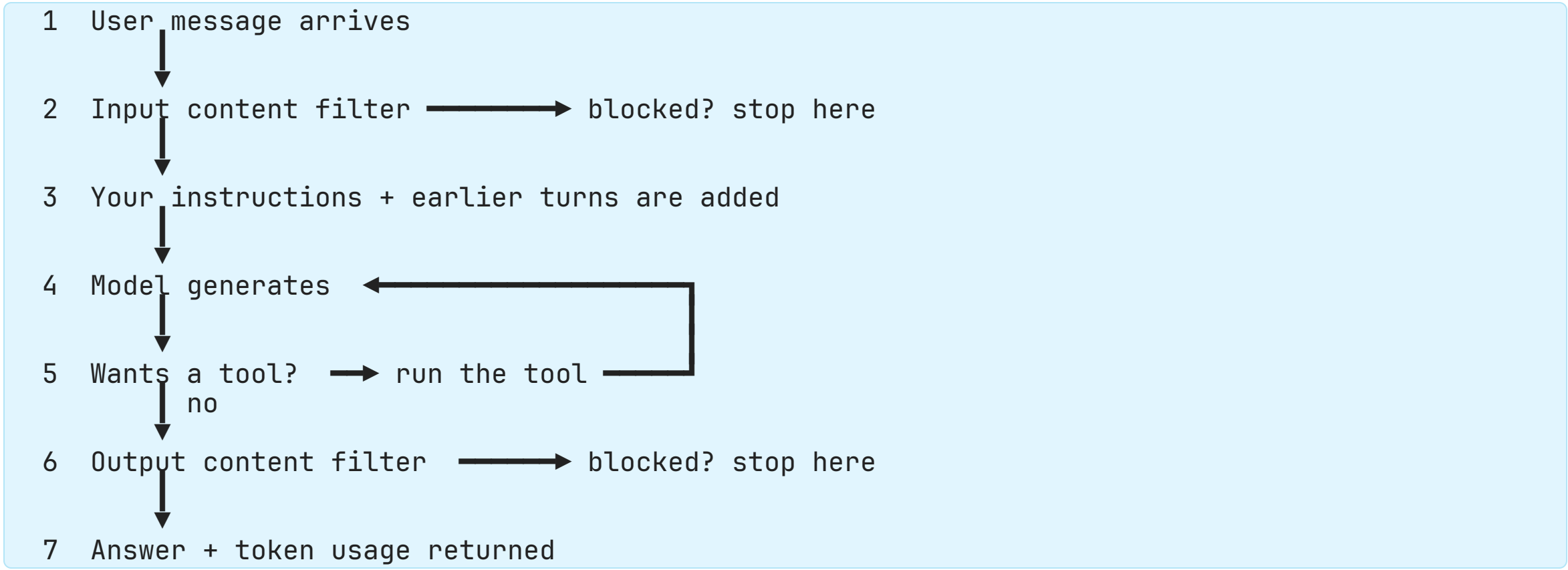
Term	In plain words
Model	The trained brain, e.g. `gpt-5.2`. You cannot call it directly
Deployment	Your named, callable copy of a model, with its own quota and filters
Endpoint	The URL your code sends requests to
Inference	One call to the model — sending text and getting text back
Token	A chunk of text (~4 characters). You pay per token, in and out
Embedding	Text turned into a list of numbers so that similar text lands close by
Grounding	Putting real source text into the prompt so the answer is based on facts
Tool	Something the model may ask your app to run: code, search, your function
Agent	A model that loops — think, call a tool, look at the result, think again

Every later slide uses these words. Come back here whenever one stops making sense.

© Rick Beerendonk

Life of One Request

Everything in this presentation happens somewhere in these seven steps.



Steps 4 and 5 form a loop. That loop is the entire difference between a chat app and an agent.

© Rick Beerendonk

Microsoft Foundry

glasspaper



© Rick Beerendonk

What is Microsoft Foundry?

Developer platform for AI solutions:

Capability	What you get
Model catalog	10,000+ models from Microsoft, OpenAI, Meta, Mistral, Hugging Face, Anthropic
Model deployment	Deploy, version, and scale models
Foundry Tools	Language, Speech, Translator, Document Intelligence, Content Understanding
Agents	Build and host AI agents with tools and knowledge
Evaluations	Test quality and safety before deployment
Tracing	Observe agent and inference activity

Recommended for all but the simplest AI solutions

glasspaper oblicum

© Rick Beerendonk

Foundry Resource and Projects

[EXAM]

Architecture:

- **Foundry resource** — top-level Azure resource; contains one or more projects
- **Project** — working space; one project is the default
- Projects contain: model deployments, agents, tools (incl. MCP), knowledge (Foundry IQ)

Endpoints:

Purpose	Endpoint format
Project (Foundry SDK)	`https://{resource-name}.services.ai.azure.com/api/projects/{project-name}`
Azure OpenAI (OpenAI SDK)	`https://{resource-name}.openai.azure.com/openai/v1`

API keys work only on the Azure OpenAI endpoint; the project endpoint takes Microsoft Entra ID.

Minimum administration to connect an app: the **endpoint URI plus a subscription key**. OAuth tokens, SAS tokens, and certificate-based managed identity all add credential setup to manage.

© Rick Beerendonk

Plan the AI Solution

Start with the capability, then choose the service:

AI capability	Typical Azure solution
Generative AI and agents	Foundry models, prompts, tools, and agents
Natural language processing	Azure Language text analysis
Computer speech	Azure Speech, audio models, and Voice Live
Computer vision	Multimodal models and Content Understanding
Information extraction	Document Intelligence and Content Understanding

Planning questions:

- What input and output modalities are required?
- Does the solution need private or current data?
- Is a deterministic API preferable to an autonomous agent?
- Which identity, data residency, safety, and latency requirements apply?

© Rick Beerendonk

Foundry Tools

Prebuilt APIs and models for predictable, focused tasks:

Tool	Common use
Azure Language	Entity extraction, sentiment, summarization, and PII
Azure Speech	Speech-to-text, text-to-speech, and real-time speech
Azure Translator	Text translation and transliteration
Azure Document Intelligence	Fields, tables, and layout from documents
Azure Content Understanding	Multimodal extraction from documents, images, audio, and video

Use a Foundry Tool when:

- The task is narrow and well-defined
- Consistent output matters more than open-ended reasoning
- Cost and latency should be predictable

glasspaper oblicum

© Rick Beerendonk

Developer Tools and SDKs

Tool	Best for
Microsoft Foundry portal	Explore models, test prompts, configure agents and tools
Visual Studio Code + Foundry Toolkit	Code-first development, YAML, source control
Microsoft Foundry SDK	Automate projects, models, agents, evaluations, and connections
OpenAI SDK	Portable chat and Responses API application code
Azure service SDKs	Focused APIs such as Language, Speech, and Document Intelligence

Recommended workflow: prototype in the portal, export or write SDK code, then test the application with the same model and deployment configuration.

© Rick Beerendonk

SDK Choice: Foundry vs OpenAI

Use Foundry SDK (`azure-ai-projects`)	Use OpenAI SDK
Agents, evaluations, tracing	Maximum OpenAI compatibility
Connections, project governance	Portability across providers
Catalog models beyond Azure OpenAI	Simple chat inference

```
pip install azure-ai-projects azure-identity openai
```


© Rick Beerendonk

AIProjectClient Setup

```python
from azure.identity import DefaultAzureCredential
from azure.ai.projects import AIProjectClient

project_client = AIProjectClient(
    endpoint="https://{resource-name}.services.ai.azure.com/api/projects/{project-name}",
    credential=DefaultAzureCredential()
)

openai_client = project_client.get_openai_client()
```

Note: `endpoint` is the first parameter, `credential` is second

© Rick Beerendonk

OpenAI SDK Direct (Azure OpenAI endpoint)

```python
from openai import OpenAI
from azure.identity import DefaultAzureCredential, get_bearer_token_provider

token_provider = get_bearer_token_provider(
    DefaultAzureCredential(), "https://ai.azure.com/.default"
)

openai_client = OpenAI(
    base_url="https://{resource-name}.openai.azure.com/openai/v1/",
    api_key=token_provider
)
```

Use when: maximum OpenAI library compatibility is required

glasspaper oblicum

© Rick Beerendonk

RBAC Roles for Inference

[EXAM]

Authentication proves *who* you are. RBAC decides *what you may do*. `DefaultAzureCredential` plus `az login` still returns **HTTP 403** without the right role.

Role	Grants
Foundry User (formerly Azure AI User)	Build in a project and call models — least privilege choice
Foundry Agent Consumer	Call agent endpoints only
Foundry Project Manager, Account Owner	Manage projects, resources, and role assignments — far too broad
Cognitive Services OpenAI User	Completion and embedding calls on an Azure OpenAI resource
Cognitive Services User	All data actions and list keys — more than inference needs
Cognitive Services Data Reader	Read data — no inference
Contributor	Full resource management — far too broad

Rule: for an app that only sends prompts to a Foundry resource, assign **Foundry User**. Roles that start with "Cognitive Services" are meant for standalone AI services resources: on an Azure OpenAI resource the least-privilege choice is **Cognitive Services OpenAI User**.

© Rick Beerendonk

Provisioning Resources

[EXAM]

Create a multi-service resource (one key and endpoint for sentiment, OCR, and future services):

Choice	Value
HTTP method	`PUT` — creates a named ARM resource (`.../accounts/CS1`)
Resource `kind`	`CognitiveServices` — multi-service (`AIServices` = Foundry)
Single-service	`TextAnalytics`, `ComputerVision` — separate key and endpoint each

`POST` is for action endpoints; `PATCH` only updates an existing resource.

Customer-managed key (CMK) encryption:

```
az cognitiveservices account create --kind OpenAI \
  --encryption '{"keySource":"Microsoft.KeyVault","keyVaultProperties":{...}}'
```

`keySource` defaults to `Microsoft.CognitiveServices` (Microsoft-managed key). Set it to `Microsoft.KeyVault` for CMK.

Deploy an image model (gpt-image-2): use **Microsoft Foundry** and the **Azure CLI**. Microsoft Graph is for Microsoft 365 data, not model deployment.

© Rick Beerendonk

Connections in Bicep

[EXAM]

A **connection** stores where a dependency lives and how to authenticate to it, so applications and agents do not carry credentials. The Key Vault connection is created on the Foundry resource.

```
resource kvConnection 'Microsoft.CognitiveServices/accounts/connections@2025-04-01-preview' = {
  name: 'kv-connection'
  parent: foundryResource
  properties: {
    category: 'AzureKeyVault'          // must match the resource class
    target: existingKeyVault.id
    authType: 'AccountManagedIdentity' // identity-based, no stored secret
  }
}
```

Property	Why this value
`category`	Identifies the connected resource class — `AzureKeyVault`
`authType`	Key Vault authenticates by **identity**, not by an account key

Trap: an account-key or API-key `authType` puts a secret back into the connection — the opposite of the requirement.

glasspaper oblicum

© Rick Beerendonk

Models

Catalog, deployment, benchmarks, and evaluation

glasspaper oblicum

© Rick Beerendonk

Model Catalog

10,000+ models — category overview:

Category	Examples
LLMs	GPT-5.2, Mistral Large, Llama 3.3 70B
SLMs	Phi-4, Phi-4-mini, Mistral Small
Reasoning	GPT-5 series (chain-of-thought is internal)
Embedding	text-embedding-3-large
Image generation	gpt-image-2, FLUX
Video generation	Sora 2
TTS / STT	gpt-4o-mini-tts, gpt-4o-mini-transcribe
Multimodal	gpt-5.2, Phi-4-multimodal-instruct

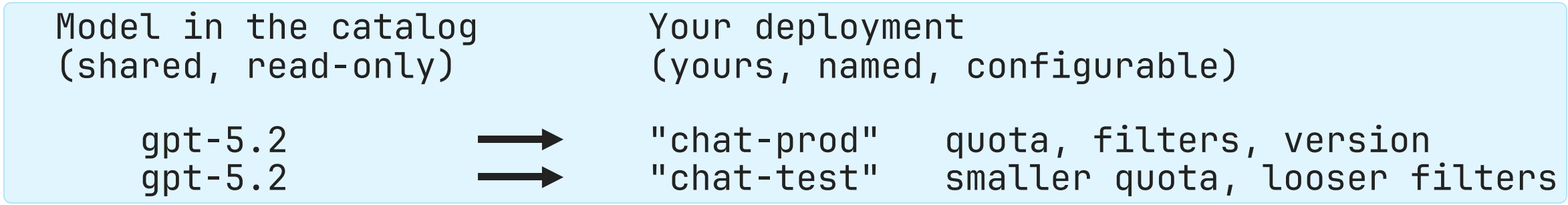
Chat vs Reasoning: Non-reasoning models benefit from chain-of-thought prompting; reasoning models handle it internally — don't add "take a step-by-step approach" to their prompts

glasspaper oblicum

© Rick Beerendonk

Model vs Deployment

You never call a model. You call **your deployment of** that model.



The deployment carries the things you control: **name, capacity, content filters, and version policy**. Two deployments of the same model can behave differently.

Why it matters: rate limits, cost, and safety settings are all per deployment — never per model.

© Rick Beerendonk

Retrieval Practice — Lab 1

Pause here and complete Lab 1, then check its solution.

glasspaper oblicum

© Rick Beerendonk

Deployment Types

[EXAM]

Type	Description	Best for
Global Standard	Auto-routing, widest availability	Default choice for most workloads
Global Provisioned (PTU)	Dedicated throughput globally	Predictable, high-volume production
Global Batch	Async, 24-hour SLA, 50% cost	Large offline batch jobs
Data Zone Standard	Data residency within geographic zone	EU/regional data compliance
Data Zone Provisioned	Dedicated + data zone	High-volume + data compliance
Data Zone Batch	Async + data zone	Batch + data compliance
Standard	Single region, no rerouting	Single-region pinning
Regional Provisioned	Dedicated, single region	Single-region + dedicated capacity
Developer	Fine-tuned model evaluation, 24 hours	Testing fine-tuned variants

Use Global Standard whenever possible for maximum capabilities

© Rick Beerendonk

Choosing a Deployment — Four Questions

[EXAM]

1. Where may the data be processed?

`Global` routes to any region with capacity — it **breaks an EU-only residency rule**. Use **Data Zone** for an EU, US, or APAC boundary, or **Standard** (single region) to pin one region.

2. Is the demand steady or variable?

`Provisioned` reserves paid throughput and is wasted on variable or low-volume traffic. Variable interactive traffic → **Standard**.

3. Does it need a real-time answer?

`Batch` is asynchronous. Real-time REST → **Standard**, which is managed and does **not consume your VM vCPU quota** (a self-hosted container does).

4. Must behavior stay stable?

Set the version-update policy to `OnceCurrentVersionExpired` so the version only changes at retirement.

© Rick Beerendonk

Choosing a Model

[EXAM]

Requirement	Choose
Deep multi-step reasoning over retrieved documents	**LLM**
Simple, high-volume, cost-sensitive tasks	SLM
Vectors for search indexing and queries	Embedding model
Topics or terms only — no generated answer	Key phrase extraction

Model cascade — do not pick one model for everything:

```
simple FAQ request   ➡  gpt-5-mini   small model: cheap, fast
complex reasoning    ➡  gpt-5.2      large model: accurate
```

A router classifies each request first. Sending everything to the smallest model loses quality; sending everything to the largest keeps cost and latency high; raising `max_output_tokens` for all requests only adds cost.

© Rick Beerendonk

Model Benchmarks — Quality

Quality Index: 0–1 average across 8 benchmarks (higher = better)

Benchmark	Tests
BIG-Bench Hard	Challenging reasoning tasks
MuSR	Multi-step reasoning
GPQA	Graduate-level science Q&A
MMLU-Pro	Multi-domain knowledge
MBPP+	Python problem solving
ChemBench	Chemistry
FrontierScience	Scientific reasoning
τ^2-bench (telecom)	Agentic tool-call selection

glasspaper oblicum

© Rick Beerendonk

Model Benchmarks — Safety, Cost, Performance

Safety:

Benchmark	Metric	Better is
HarmBench	ASR (Attack Success Rate)	Lower
ToxiGen	F1 (toxicity detection)	Higher
WMDP	Hazardous knowledge accuracy	Lower

Cost: in USD — lower is better

Performance: Latency mean and percentiles (P50–P99), time to first token (TTFT), Throughput (generated and total tokens per second)

© Rick Beerendonk

Evaluating a Model

[EXAM]

Evaluation targets: Model | Agent | Dataset

Dataset options: Upload new CSV/JSONL, Use existing, Generate synthetic (specify topic + row count)

Quality metrics (AI-assisted):

Metric	Description
Groundedness	Is every claim supported by the context?
Groundedness Pro	Binary: grounded or not
Relevance	Does the response answer the question?
Coherence	Is it logically structured?
Fluency	Is the language natural and correct?

Safety metrics: severity score 0–7 per response; above the threshold (default 3) it fails. Defect rate = share of responses that fail
Evaluators: Self-harm, Hateful & unfair, Violent, Sexual, Protected material, Indirect attack

NLP metrics (no evaluator LLM needed): F1, BLEU, METEOR, ROUGE, GLEU

glasspaper oblicum

© Rick Beerendonk

Chat Application

glasspaper

oblicum

© Rick Beerendonk

Responses API (Recommended for New Development)

[EXAM]

```
response = openai_client.responses.create(
    model="gpt-5.2",
    input="What is the capital of France?",
    instructions="You are a helpful assistant.",
    reasoning={"effort": "low"},
    max_output_tokens=200,
)

print(response.output_text)
print(response.id)              # for multi-turn chaining
print(response.status)
print(response.usage.total_tokens)
```

Works with: Azure OpenAI models and other Foundry Models sold by Azure (DeepSeek, Mistral)

© Rick Beerendonk

Responses API — Multi-Turn

[EXAM]

Option 1: Chain by response ID (server manages history)

```
response2 = openai_client.responses.create(
    model="gpt-5.2",
    input="What about Paris specifically?",
    instructions="You are a helpful assistant.",  # not carried over: send again
    previous_response_id=response1.id,
)
```

Option 2: Manual conversation history

```
messages = [
    {"role": "system", "content": "You are a helpful assistant."},
    {"role": "user", "content": "What is the capital of France?"},
    {"role": "assistant", "content": response1.output_text},
    {"role": "user", "content": "Tell me more."},
]
response2 = openai_client.responses.create(model="gpt-5.2", input=messages)
```

glasspaper oblicum

© Rick Beerendonk

Durable Conversation State

[EXAM]

Use a **conversation** when an agent must reuse complete interaction history across turns and sessions. A durable conversation ID links messages, tool calls, and tool outputs for the ongoing case.

Do not confuse: a response is one model result; an output item is part of a result; an agent is the reusable definition. The conversation is the durable runtime context.

© Rick Beerendonk

Persisting Conversation History

Manual history is gone when the session ends, and a stored response behind `previous_response_id` is kept for 30 days by default. For multi-session or multi-user apps, **persist the messages array externally**.

```python
import json

# End of session — save
history = [
    {"role": "developer", "content": "System instructions..."},
    {"role": "user",      "content": user_input},
    {"role": "assistant", "content": response.output_text},
]
db.save(user_id, json.dumps(history))

# Start of next session — reload
history = json.loads(db.load(user_id))
history.append({"role": "user", "content": new_input})
response = openai_client.responses.create(model="gpt-5.2", input=history)
history.append({"role": "assistant", "content": response.output_text})
db.save(user_id, json.dumps(history))
```

Storage options:

Store	Best for
Cosmos DB	Per-user history, scalable, JSON-native
Azure Managed Redis	Short-lived sessions, fast read/write
Azure SQL / PostgreSQL	Structured history with user metadata

© Rick Beerendonk

Streaming

```
stream = openai_client.responses.create(
    model="gpt-5.2",
    input="Tell me a long story.",
    stream=True,
)
for event in stream:
    if event.type == "response.output_text.delta":
        print(event.delta, end="")
```


© Rick Beerendonk

ChatCompletions API (Portability)

```
response = openai_client.chat.completions.create(
    model="gpt-5.2",
    messages=[
        {"role": "system", "content": "You are a helpful assistant."},
        {"role": "user", "content": "What is the capital of France?"},
    ]
)
print(response.choices[0].message.content)
```

Use when: maximum portability or OpenAI compatibility is required

	Responses API	ChatCompletions
Recommended for	New development	Portability
Multi-turn	`previous_response_id`	Manual history array
Result	`response.output_text`	`response.choices[0].message.content`

glasspaper oblicum

© Rick Beerendonk

Retrieval Practice — Lab 2

Pause here and complete Lab 2, then check its solution.

glasspaper oblicum

© Rick Beerendonk

Tools

glasspaper



© Rick Beerendonk

Tools in the Responses API

[EXAM]

Specify tools in `responses.create()` — model decides when to use them:

Tool	Type name	Capability
Code interpreter	`code_interpreter`	Sandboxed Python runtime
Web search	`web_search`	Live web retrieval
File search	`file_search`	Vector search over uploaded files
Function	`function`	Custom application logic

```
response = client.responses.create(
    model=DEPLOYMENT,
    input=[{"role": "user", "content": "What is the square root of 16?"}],
    tools=[{"type": "code_interpreter", "container": {"type": "auto"}}]
)
print(response.output_text)  # "The square root of 16 is 4."
```

Note: "Foundry Tools" (Language, Speech, etc.) are Azure AI APIs — distinct from these prompt tools

© Rick Beerendonk

`code_interpreter` and `web_search`

[EXAM]

`code_interpreter` — model generates and runs Python in a sandbox:

- Pre-installed: `pandas`, `numpy`, `math`, `matplotlib`
- Can read/write files (CSV, JSON, images); auto-corrects errors
- No network access; timeout and memory limits apply
- Many models internally call this the *python tool*

`web_search` — model retrieves current information from the web:

- Use type name `"web_search"` (formerly `"web_search_preview"`)
- Runs on Bing; the **Grounding with Bing Search** tool `"bing_grounding"` in Foundry agents does the same through a Bing connection
- Model decides when to search; issues query automatically
- Produces source-grounded answers for time-sensitive topics
- Adds latency and token usage; source quality varies

© Rick Beerendonk

`file_search` and `function`

[EXAM]

`file_search` — semantic search over uploaded documents:

```
vector_store = client.vector_stores.create(name="policy-docs")
client.vector_stores.files.upload_and_poll(
    vector_store_id=vector_store.id, file=open("policy.pdf", "rb")
)
response = client.responses.create(
    model=DEPLOYMENT, input=messages,
    tools=[{"type": "file_search", "vector_store_ids": [vector_store.id]}]
)
```

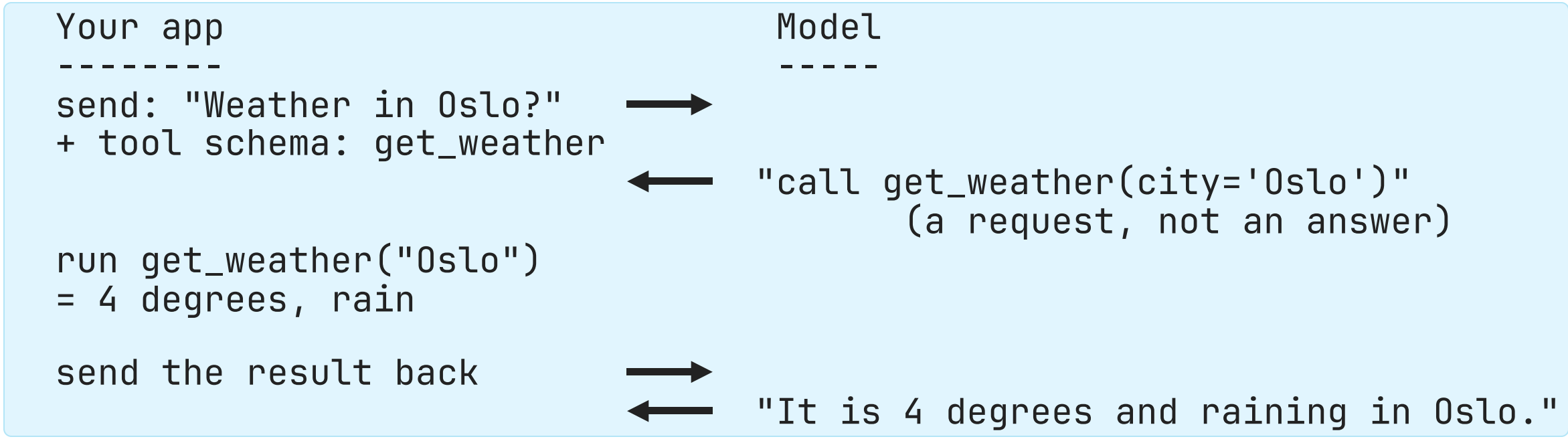
`function` — your own code as a tool. The model never runs it; it asks you to.

Enterprise-scale multi-source knowledge: use Foundry IQ agents instead of `file_search`

© Rick Beerendonk

Function Calling — Who Does What, When

The most common misunderstanding: the model does **not** execute your function.



Two round trips, always. The model chooses *whether* and *with which arguments*; your code stays in control of *what actually runs*.

glasspaper oblicum

© Rick Beerendonk

Retrieval Practice — Lab 3

Pause here and complete Lab 3, then check its solution.

glasspaper oblicum

© Rick Beerendonk

Optimization

glasspaper

oblicum

© Rick Beerendonk

Optimization Sequence

[EXAM]

Problem: Model lacks private/current data or doesn't respond in the right style

```
cheap, minutes          1  Prompt engineering   change what you *say*
       |                2  RAG                  change what it *knows*
costly, days            3  Fine-tuning          change what it *is*
```

Climb only one rung at a time. Most problems that look like "the model is wrong" are solved on rung 1 or 2.

Always establish a baseline before moving to the next step — otherwise you cannot tell whether the step helped.

© Rick Beerendonk

Prompt Components and Patterns

[EXAM]

Components: System message, User message, Assistant message, Examples

Patterns:

Pattern	Description
Persona	"You are a seasoned marketing professional..."
Format template	Explicit output structure (bullets, JSON schema)
Chain-of-thought	"Take a step-by-step approach" (non-reasoning models only)
Few-shot	Input/output examples: `"Good" → "positive"`, `"Bad" → "negative"`
	(one-shot = `1` example, zero-shot = 0 examples)

Use delimiters (`---`, Markdown headings, XML tags) to separate sections; watch for recency bias

Restricting scope (e.g. "answer only about Contoso products") belongs in the **system message**. Few-shot examples demonstrate a pattern; they do not enforce a boundary.

© Rick Beerendonk

Model Parameters

[EXAM]

Reasoning effort: how much the model thinks before it answers
Output token limit: the most tokens it may generate, reasoning included

Control	Low value	High value
Reasoning effort	Faster and cheaper — simple summaries	Slower and costlier — hard problems
`max_output_tokens`	Short answer, may be cut off at the limit	Room for reasoning and a full answer

Where to set them: `reasoning={"effort": "low"}` and `max_output_tokens` in the Responses API; `reasoning_effort="low"` and `max_completion_tokens` in Chat Completions. Effort values include `none`, `low`, `medium`, and `high`; which ones are accepted differs per model.

Reasoning models are controlled with the reasoning effort; with reasoning switched on, `temperature` and `top_p` are rejected.

Cut off vs incomplete: raise `max_output_tokens` when the answer stops at the limit. It does **not** add content the model never wrote.

glasspaper oblicum

© Rick Beerendonk

Fixing Incomplete Output

[EXAM]

Problem: the response keeps omitting required clauses.

No parameter can add content that was never generated:

- ❌ Lower `max_output_tokens` — truncates even more
- ❌ Switch model without measuring — unproven
- ❌ Block the failing response — detects the gap, does not fill it

Solution: a retry evaluation (reflection pass) in application logic, before returning

```
answer = generate(prompt)
review = evaluate(answer, required_clauses)
if review.missing:
    answer = generate(prompt, feedback=review.missing)
return answer
```

Raising `max_output_tokens` is a valid **supporting** fix — it permits a complete answer, but it does not produce one.

glasspaper oblicum

© Rick Beerendonk

RAG — Retrieval Augmented Generation

Problem: Models lack private/current data → hallucinate

Pattern: Retrieve → Augment → Generate

- **Retrieve:** Search an index for relevant documents
- **Augment:** Add retrieved content to the prompt as context
- **Generate:** Model responds grounded in retrieved documents

How it works:

- **Embeddings:** convert text into numbers. Similar meanings produce nearby vectors.
- **Azure AI Search:** keyword search finds exact words; vector search finds similar meaning. **Hybrid search combines both — recommended for RAG.**

© Rick Beerendonk

RAG in Slow Motion

The model is never retrained. You simply hand it the right page before it answers.

```
Ahead of time (once)
  documents  ➡  split into chunks  ➡  embed  ➡  store in index

At question time (every request)
1  user asks     "How many holiday days do I get?"
2  embed the question, search the index
3  get back 3 chunks of the real HR handbook
4  build the prompt:
       "Answer using ONLY this context: &lt;3 chunks&gt;
        Question: How many holiday days do I get?"
5  model answers from the context, with citations
```

Key insight: steps 1–3 are search, step 5 is the model. RAG quality is usually a *search* problem, not a model problem.

glasspaper oblicum

© Rick Beerendonk

Two Types of RAG — Classic vs Agentic

Aspect	Classic RAG (single-shot)	Agentic RAG (Foundry IQ retrieval engine)
Retrieval calls	**One** search per user turn	**Many** — the agent decides when and what to search again
Query rewriting	None (uses the user question verbatim)	Agent rewrites, decomposes, and retries failed queries
Multi-source	Typically one index	Federated across knowledge sources, picks the right one per step
Cost / latency	Low — 1 search + 1 generate	Higher — multiple search/reason iterations
When to choose	FAQ, single document set, deterministic answer	Complex questions, multi-hop reasoning, mixed sources

"Generative RAG" / agentic retrieval is the same idea: the LLM is given retrieval as a *tool* (`file_search`, AI Search) and runs its own loop — search → read → decide → maybe search again — instead of a fixed retrieve-then-generate pipeline.

glasspaper oblicum

© Rick Beerendonk

RAG — Groundedness Gate

[EXAM]

Problem: Retrieved documents may not contain the needed information → model fills the gap with training data → hallucination

Solution: Verify context is sufficient before generation

Option A — LLM gate (pre-generation):

```
check = openai_client.responses.create(
    model="gpt-5.2",
    instructions="Answer only YES or NO.",
    input=f"Does this context contain enough information to answer: '{query}'?\n\nContext:\n{context}"
)
if "YES" in check.output_text.upper():
    answer = generate_answer(context, query)
else:
    answer = "I don't have that information in the company documents."
```

Option B — Groundedness evaluator (post-generation):

```
from azure.ai.evaluation import GroundednessEvaluator

evaluator = GroundednessEvaluator(model_config)
score = evaluator(query=query, context=context, response=answer)
if score["groundedness"] < 3:
    return "I cannot answer reliably from the available documents."
```

Option A **blocks** generation; Option B generates first, then filters

glasspaper oblicum

© Rick Beerendonk

Fine-Tuning

When to fine-tune: consistent style/tone, specific output format, reduce prompt length, distillation, enhance tool usage

Technique: LoRA (Low-Rank Adaptation) — updates a smaller subset of parameters; faster and cheaper than full retraining

Types:

Type	Description
SFT	Supervised Fine-Tuning — prompt/response pairs
RFT	Reinforcement Fine-Tuning — grader rewards correct responses
DPO	Direct Preference Optimization — preferred vs. non-preferred pairs (lighter than RL)

Can combine methods (e.g., SFT first, then DPO for alignment)

© Rick Beerendonk

Fine-Tuning — Training Data Format

JSONL format — one example per line:

```
{"messages": [{"role": "system", "content": "You are a helpful assistant."}, {"role": "user", "content": "What is the capital of France?"}, {"role": "assistant", "content": "Paris."}]}
{"messages": [{"role": "system", "content": "You are a helpful assistant."}, {"role": "user", "content": "What is 2 + 2?"}, {"role": "assistant", "content": "4."}]}
```

Always establish a baseline before fine-tuning

glasspaper oblicum

© Rick Beerendonk

Retrieval Practice — Lab 4

Pause here and complete Lab 4, then check its solution.

glasspaper oblicum

© Rick Beerendonk

Responsible AI

glasspaper

oblicum

© Rick Beerendonk

The Six Responsible AI Principles

[EXAM]

Principle	Question it answers
Fairness	Are all groups treated equitably?
Reliability & safety	Does it behave dependably, including under stress?
Privacy & security	Is user data protected?
Inclusiveness	Can people of all abilities use it?
Transparency	Do users understand how it works and how their data is used?
Accountability	Is a named person answerable for the outcome?

Trap: "tell users how their data is processed" → **Transparency**. Not fairness (equitable treatment), inclusiveness (accessibility), or reliability and safety (dependable operation).

glasspaper oblicum

© Rick Beerendonk

The 4-Stage Process

Map → Measure → Mitigate → Manage

Stage	Goal
Map	Identify and prioritize potential harms
Measure	Quantify the presence of harms in your system
Mitigate	Reduce harms at multiple layers
Manage	Deploy and operate responsibly

glasspaper oblicum

© Rick Beerendonk

Map Stage

1. **Identify harms** — by scenario, user type, potential misuse
2. **Prioritize** — by likelihood × impact
3. **Test/verify** — red team: deliberately probe for failures
4. **Document and share** — record findings for ongoing reference

glasspaper oblicum

© Rick Beerendonk

Measure Stage

1. **Prepare** adversarial measurement prompts
2. **Submit** prompts to the model
3. **Apply evaluation criteria** — manual review first, then automate

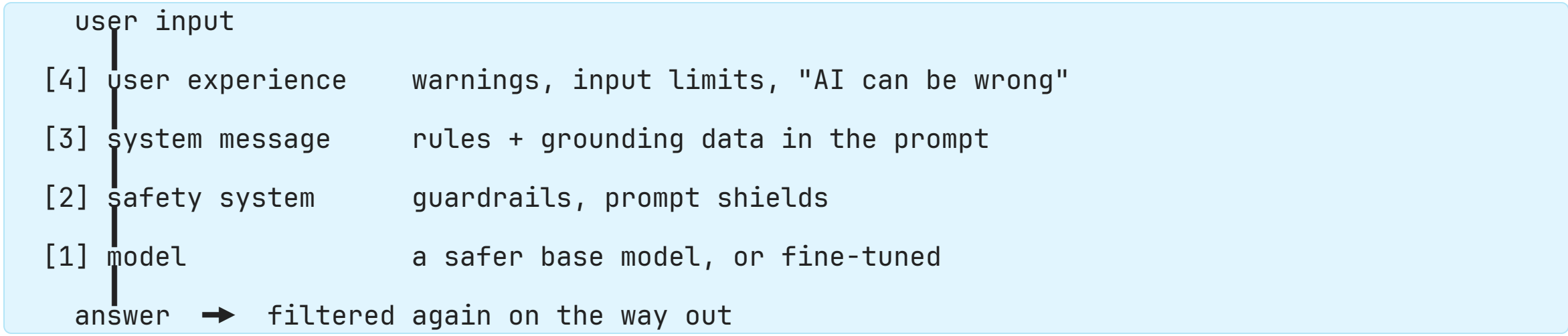
Start manual, then automate — automation requires validated criteria

glasspaper oblicum

© Rick Beerendonk

Mitigate Stage — 4 Layers

No single layer stops everything. They are checkpoints a request passes through.



Layer	Description
1. Model layer	Select an appropriate model; fine-tune for safer behavior
2. Safety system	Guardrails (input/output) + prompt shields + protected material
3. System message & grounding	Behavioral parameters + RAG grounding
4. User experience	UI controls, input validation, transparency

glasspaper oblicum

© Rick Beerendonk

Configure Guardrails

[EXAM]

A **guardrail** (formerly content filter) is a set of **controls**, assigned per deployment under **Build → Guardrails** in the Foundry portal. Each control has a **risk**, **intervention points**, and an **action** (annotate or block).

Content harm controls — user input and output

Applied to user prompts (input) and model completions (output) separately:

Category	Configurable blocking level
Hate & fairness	Low / **Medium** / High
Sexual	Low / **Medium** / High
Violence	Low / **Medium** / High
Self-harm	Low / **Medium** / High

Default: blocking level Medium blocks medium and high severity — content at Low or Safe passes through

© Rick Beerendonk

Prompt Shields and Protected Material

[EXAM]

Prompt Shields (input)

Type	Default	Purpose
User prompt attacks	On	Detect jailbreak attempts targeting the system prompt
Indirect attacks	Off	Detect malicious instructions embedded in **documents** and other external text

Indirect attack = Cross-Domain Prompt Injection: an attacker hides instructions inside content your agent processes (e.g., a malicious PDF, text extracted from an image). Enable this when agents use RAG, File Search, or OCR text from images.

Protected material (output)

Type	Default	Purpose
Text	On	Block known copyrighted content (lyrics, recipes)
Code	On	Detect code that matches public GitHub repositories

glasspaper oblicum

© Rick Beerendonk

Where to Moderate

[EXAM]

An agent has **four** content checkpoints, not two:

```
user input -[1]-> model -[2]-> tool call -> tool -[3]-> tool response -> model -[4]-> output
```

Moderating only user input and final output leaves the **tool call** and **tool response** unchecked.

Input surface	What actually protects it
Direct user message	**User prompt shield**
Text extracted from a document	**Document prompt shield** (indirect attacks)
Uploaded image	Extract its text with OCR, then use the **document prompt shield**

Traps: image moderation detects harmful *visuals*, not hidden instructions. Protected-material detection flags copyrighted content, not injection.

© Rick Beerendonk

Blocking Levels and Spotlighting

[EXAM]

A blocking level names the strictness, not the severity it blocks:

```
Low      blocks High only              ← least restrictive
Medium   blocks Medium + High          ← default
High     blocks Low + Medium + High    ← most restrictive, most rejections
```

Raising a control to **High** *increases* rejections — it also blocks low-severity content (formerly a severity threshold, where Low was the strictest). **Protected material** is a text match, so it ignores blocking levels entirely and its result is the same at every level.

Spotlighting (preview, Chat Completions API only) marks external content (OCR text, retrieved documents) as untrusted **data**, so the model does not follow instructions found inside it.

Requirement	Setting
Stop detected prompt attacks	Prompt shields → **Block**
Lower the model's trust in external text	Enable **spotlighting**

Both are needed: shields without spotlighting do not lower trust; spotlighting without Block does not stop the attack.

© Rick Beerendonk

Retrieval Practice — Lab 5

Pause here and complete Lab 5, then check its solution.

glasspaper oblicum

© Rick Beerendonk

Manage Stage

Pre-release reviews: Legal, Privacy, Security, Accessibility

Release and operate:

- **Phased delivery plan** — release to restricted group first; gather feedback
- **Incident response plan** — estimate time to respond to unanticipated incidents
- **Rollback plan** — steps to revert to previous state
- **Block capability** — immediately block harmful responses when discovered
- **User feedback** — allow reporting of inaccurate, harmful, or offensive content
- **Telemetry** — track user satisfaction; comply with privacy laws

© Rick Beerendonk

Retrieval Practice — Lab 6

Pause here and complete Lab 6, then check its solution.

glasspaper oblicum

© Rick Beerendonk

Operate and Secure AI Solutions

The production concerns that complete a generative AI application.

glasspaper oblicum

© Rick Beerendonk

Problem: It Worked in the Playground

The demo was approved on Friday. On Monday it is real software.

- ❌ The key is in a config file that three teams can read
- ❌ Anyone on the internet can reach the endpoint
- ❌ At 09:00 every request returns `429 Too Many Requests`
- ❌ A prompt was changed and nobody knows by whom, or when
- ❌ A customer asks why the model said that — and there is no record

Need: the same discipline you already apply to any production system, translated to AI.

glasspaper oblicum

© Rick Beerendonk

Five Questions Every Production AI System Must Answer

1	WHO can call it?	identity	Entra ID, managed identity, RBAC
2	FROM WHERE?	network	private endpoint, network rules
3	HOW MUCH can it use?	capacity	RPM, TPM, PTU, token cost
4	HOW does a change ship?	delivery	source control, pipeline, rollback
5	WHAT actually happened?	oversight	metrics, traces, evaluation, audit

The rest of this section takes the five questions in order.

Exam habit: when a question describes a production failure, first decide *which of the five* it belongs to. That usually eliminates three of the four answers.

© Rick Beerendonk

Identity: Keys, Tokens, and Managed Identity

[EXAM]

Key-based authentication: simple to start with, but secrets must be stored and rotated securely.

Microsoft Entra ID: preferred for production applications.

Managed identity: Azure manages the application identity; no client secret is stored in code.

Exam decision pattern:

Scenario	Preferred approach
Local development	`DefaultAzureCredential` with developer login
Azure-hosted application	System-assigned or user-assigned managed identity
Temporary external client	Short-lived token with narrowly scoped permissions
Legacy or isolated integration	Key stored in Key Vault and rotated

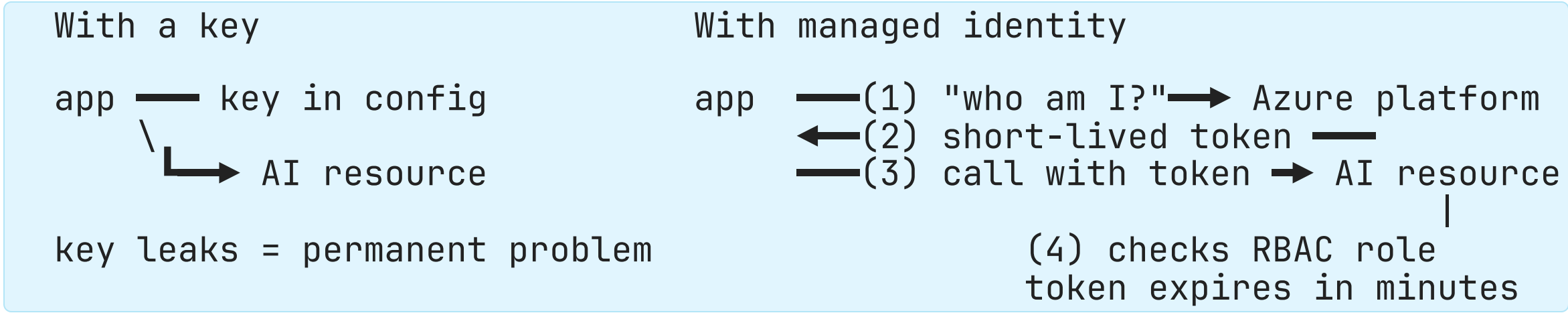
Keyless does not mean permissionless: assign the identity only the required Azure roles.

Least-privilege role	Grants
Foundry User	Call models on a Foundry resource
Key Vault Secrets User	Read secrets — not Administrator
Storage Blob Data Reader	Read blobs — not Contributor (write/delete)

© Rick Beerendonk

How Keyless Actually Works

A key is something you *have*. An identity is something you *are* — and Azure can vouch for it.



`DefaultAzureCredential` hides this. Locally it uses your developer login; in Azure it uses the managed identity — same code, no secret either way.

Keyless does not mean permissionless: without an RBAC role assignment, step 4 fails with 403.

© Rick Beerendonk

Private Networking and Data Protection

[EXAM]

Private networking controls how traffic reaches the AI resource.

- Private endpoint: expose the resource through a private IP in a virtual network
- Public network access: disable when all clients should use private connectivity
- Network rules: restrict allowed subnets and trusted services
- Egress controls: limit where retrieved documents and tool calls can go
- Data residency: select an appropriate region or data zone deployment

Exam distinction: authentication answers *who may call*; networking answers *from where the resource may be reached*.

Also consider: encryption, Key Vault, diagnostic settings, retention, redaction, and tenant isolation.

© Rick Beerendonk

Retrieval Practice — Lab 7

Pause here and complete Lab 7, then check its solution.

glasspaper oblicum

© Rick Beerendonk

Quotas, Scaling, and Cost

[EXAM]

Capacity is not one number. Three separate limits can each stop you.

```
RPM  requests per minute   how OFTEN you may call     burst of calls → 429
TPM  tokens per minute     how MUCH text per minute   long prompts   → 429
PTU  provisioned units     capacity you RESERVED      utilization cap, fixed bill
```

A token is roughly four characters. A 2-page prompt is ~1,000 tokens, so 60 such calls a minute costs 60,000 TPM — even though it is only 60 requests.

Trap: `max_output_tokens` is a cap, not a reservation. TPM counts the tokens actually sent and generated — asking for 4,000 output tokens costs nothing extra when the model replies with one word.

© Rick Beerendonk

Capacity Dimensions and Levers

Capacity has multiple dimensions:

Concern	What to monitor or choose
RPM	Requests per minute; bursts can cause 429 responses
TPM	Tokens per minute; input plus generated output tokens
PTU	Reserved throughput for predictable high-volume workloads
Concurrent jobs	Service-specific limits, such as video generation
Regional capacity	Availability and failover options
Token cost	Input, output, cached, and tool-related tokens

Optimization levers:

- Set the smallest useful output limit
- Stream long responses where appropriate
- Cache stable prompts and retrieval results
- Use a smaller model for classification or routing
- Batch offline work when latency is not important
- Spread load across deployments or regions
- Establish a baseline before changing the model or prompt

© Rick Beerendonk

Rate Limits and 429 Errors

[EXAM]

Units: TPM (Tokens Per Minute) and RPM (Requests Per Minute) — per deployment

Why you get 429 even when usage looks low:

- Standard deployments share capacity — under heavy demand the service temporarily lowers your effective limit
- RPM window is 1–10 seconds — a burst triggers it even if per-minute total is fine

Response headers (always present):

Header	When present	Meaning
`x-ratelimit-remaining-requests`	Every response	Remaining RPM budget
`x-ratelimit-remaining-tokens`	Every response	Remaining TPM budget
`retry-after-ms`	429 only	Wait time in milliseconds

HTTP 400 — request too large: move the bulk content into files and retrieve it with `file_search` instead of inlining it in the prompt.

© Rick Beerendonk

Handling 429 — Retry and Best Practices

[EXAM]

SDK built-in retry: `OpenAI(max_retries=5)` — default is 2; exponential backoff + `retry-after` header

Retry strategy: exponential backoff with jitter. Immediate retries hit the same limit; identical backoff across clients re-synchronizes them into a new burst.

Best practices:

- Keep prompts short and output **no longer than the scenario needs** — TPM counts the tokens actually sent and generated, not the configured `max_output_tokens`
- Spread requests evenly — avoid sending all requests in the same 1-second window
- Distribute load across multiple deployments or regions
- Use **Provisioned (PTU)** deployments for stress testing — reserved throughput, throttled on utilization instead of a TPM quota

© Rick Beerendonk

CI/CD and Environment Promotion

[EXAM]

Essential: treat prompts, agent definitions, tool schemas, and evaluation datasets as versioned application assets.

- Use separate development, test, and production deployments
- Run quality and safety evaluations before promotion
- Keep endpoints, keys, resource IDs, and deployment names in secure environment configuration
- Monitor releases and retain a rollback path

© Rick Beerendonk

Model Lifecycle and Retirement

[EXAM]

5 stages: Preview → GA → Legacy → Deprecated → **Retired**

Stage	Description
Preview	Early access; may change; 30-day retirement notice
GA	Generally available; 18-month lifecycle
Legacy	Replaced by newer version; still accessible
Deprecated	Blocked for new customers; existing customers can continue
Retired	**HTTP 410 Gone** — calls fail completely

Timeline (GA): At month 12 → Deprecated; at month 18 → Retired
Notifications: 60 days (GA), 30 days (Preview) — email to subscription owners + Azure Service Health (filter: "Azure OpenAI Service")

Auto-upgrade: Global Standard, Data Zone Standard, and Standard only — Provisioned must migrate manually

`versionUpgradeOption` **property:**

Value	Behavior
`OnceCurrentVersionExpired`	Upgrade only when current version retires (safest)
`OnceNewDefaultVersionAvailable`	Upgrade when any new default version is released
`NoAutoUpgrade`	Never auto-upgrade — deployment dies at retirement

API terminology trap: `lifecycleStatus: "Deprecated"` in REST API = **Retired** (410); `"Deprecating"` = Deprecated (still works)

glasspaper oblicum

© Rick Beerendonk

Retrieval Practice — Lab 8

Pause here and complete Lab 8, then check its solution.

glasspaper oblicum

© Rick Beerendonk

Monitor Your AI Application

[EXAM]

Azure Monitor collects metrics and logs automatically for all AI resources.

Two data paths:

Data	Collection	Where stored	Action needed
Platform metrics	Automatic	Azure Monitor metrics DB	None
Resource logs	Not automatic	Requires diagnostic setting	Create setting
Activity log	Automatic	Separate store	None

Dashboards (Foundry portal → Metrics): HTTP Requests, Tokens-Based Usage, PTU Utilization, Fine-tuning

Application Insights: use for application-layer tracing, custom events, request/dependency tracking

glasspaper oblicum

© Rick Beerendonk

Resource Logs, KQL, and Alerts

[EXAM]

Enable resource logs:

1. Azure portal → your AI resource → **Diagnostic settings** → **Add**
2. Select log categories → route to **Log Analytics workspace**

Query logs with KQL:

```
AzureDiagnostics | take 100
| project TimeGenerated, OperationName, DurationMs, ResultSignature

AzureMetrics | take 100
| project TimeGenerated, MetricName, Total, Average, UnitName
```

Alerts: metric alerts (token usage thresholds), log alerts (KQL-based), activity log alerts (resource changes)

© Rick Beerendonk

Which Signal Answers Which Question

[EXAM]

Symptom	Signal to read
Service-side outage	**Azure OpenAI AvailabilityRate**
HTTP 429 under load	AvailabilityRate **and Provisioned-managed Utilization V2**
Unexplained cost spike	**Processed Prompt Tokens** and **Generated Completion Tokens**
Low-quality or wrong answers	`RequestResponse` diagnostic logs
Ordered calls inside one run	**Tracing**

Diagnostic log categories:

`RequestResponse` holds the request payloads, status codes, and completions. The other two categories are `Audit` and `Trace`.

Traps: latency measures time, run success rate measures completion, evaluation metrics measure quality — none of them explain cost.

© Rick Beerendonk

Tracing Configuration

[EXAM]

Instrument with **OpenTelemetry**, collect and analyze in **Application Insights**. A Log Analytics workspace stores logs but does not instrument the app.

Requirement	Setting
Keep each service separate in the trace view	A distinct `OTEL_SERVICE_NAME` per service
Keep prompts and completions out of spans	`enable_content_recording=False`
Audit nested operations	**Hierarchical spans** — not flat log entries
Audit tool invocations	**Tool-call attributes** — not generic messages
Trace your own client code (preview)	`AIProjectInstrumentor().instrument()`

For an agent: enable **Application Insights for the agent** in the Foundry project. Updating the agent version, restarting it, or attaching a Log Analytics workspace alone sends no telemetry.

© Rick Beerendonk

Observability and Evaluation in Production

[EXAM]

Collect signals at three levels:

Level	Signals
Application	Requests, failures, dependencies, user feedback
Model	Tokens, latency, throughput, model version, rate limits, retries
AI quality	Groundedness, relevance, coherence, fluency, safety, refusal rate

Tracing should connect: user request → prompt → model response → retrieval → tool calls → final answer.

Drift means behavior or data changes over time:

- Input distribution changes
- Retrieval relevance declines
- Model version changes
- Groundedness or safety scores regress
- User feedback changes

Use evaluation datasets and alerts to detect regression before changing production behavior.

© Rick Beerendonk

Retrieval Practice — Lab 9

Pause here and complete Lab 9, then check its solution.

glasspaper oblicum

© Rick Beerendonk

Security Posture — Defender for Cloud and Sentinel

Azure Monitor / App Insights → *performance & cost* signals (latency, tokens, errors)
Microsoft Defender for Cloud + Microsoft Sentinel → *security & threat* signals for the AI workload

Service	Role for AI workloads
Defender for Cloud — AI services	Threat detection on Azure OpenAI / Foundry: prompt-injection alerts, data-leak detection, anomalous access on the resource
Defender for Cloud — secure score	Continuous configuration assessment: keyless auth, private endpoints, diagnostic settings on
Microsoft Sentinel (SIEM/SOAR)	Aggregates Defender alerts + diagnostic logs across resources; correlation rules; automated playbooks for incident response

Wiring it up:

1. Enable **Diagnostic settings** on the AI resource → Log Analytics workspace
2. Onboard the workspace to **Microsoft Sentinel**
3. Enable the **Defender for AI Services** plan in Defender for Cloud on the subscription
4. Sentinel ingests Defender alerts + `AzureDiagnostics` logs → unified detection across model abuse, key misuse, exfiltration

Exam framing: "health of the system or the data" in a security sense → Defender for Cloud (resource posture) and Sentinel (SIEM correlation), not Azure Monitor.

© Rick Beerendonk

Audit, Provenance, and Oversight

[EXAM]

Audit record: who requested the action, which model and prompt version ran, which tools and data were used, what the model returned, and which human approved it.

Provenance: retain citations, document IDs, source timestamps, model version, and evaluation version so an answer can be explained later.

Approval controls:

- Require approval before high-impact or irreversible tool calls
- Restrict tools by agent, user, environment, and task
- Escalate low-confidence decisions to a person
- Log both the requested action and the approved action
- Support block, rollback, and incident-response procedures

Exam takeaway: autonomy must be bounded by identity, permissions, tools, monitoring, and human oversight.

© Rick Beerendonk

Retrieval Practice — Lab 10

Pause here and complete Lab 10, then check its solution.

glasspaper oblicum

© Rick Beerendonk

Key Takeaways

1. **Foundry resource → Projects**; use project endpoint for Foundry SDK, Azure OpenAI endpoint for OpenAI SDK
2. **10,000+ models** — 9 deployment types; use Global Standard by default
3. **Responses API** recommended for new development; ChatCompletions for portability
4. **Cross-session history:** stored responses behind `previous_response_id` expire (30 days by default) — persist messages array externally (Cosmos DB, Redis)
5. **Tools:** `code_interpreter`, `web_search`, `file_search`, `function` — model decides when to call
6. **Optimization sequence:** Prompt engineering → RAG → Fine-tuning
7. **Fine-tuning techniques:** LoRA; types: SFT, RFT, DPO
8. **Incomplete output:** fix with a retry/reflection evaluation, not with a parameter
9. **RAG groundedness gate:** verify retrieved context is sufficient before generating — LLM gate (pre) or `GroundednessEvaluator` (post)
10. **Responsible AI:** six principles (Transparency answers "how is my data used?"); Map → Measure → Mitigate (4 layers) → Manage
11. **Moderate four points:** user input, output, tool call, tool response; **High blocking level = most restrictive**; spotlighting + Block for injected instructions
12. **Identity:** prefer Entra ID and managed identity; keyless still requires RBAC — **Foundry User** on a Foundry resource
13. **Network:** private endpoints and network rules control reachability
14. **Capacity:** RPM, TPM, PTU, concurrency, region, and token cost are separate concerns
15. **Rate limits:** TPM + RPM per deployment; retry with **exponential backoff + jitter**; TPM counts actual tokens, not `max_output_tokens`
16. **Delivery:** version AI configuration and promote it through tested environments
17. **Model retirement:** Retired = **HTTP 410**; Standard types auto-upgrade; Provisioned must migrate manually
18. **Monitoring:** combine application, model, and AI-quality signals
19. **Monitor:** platform metrics auto-collected; `RequestResponse` logs need a diagnostic setting; Provisioned-managed Utilization V2 explains 429
20. **Tracing:** OpenTelemetry + Application Insights; distinct `OTEL_SERVICE_NAME` per service; `enable_content_recording=False`
21. **Governance:** preserve provenance and require approval for high-impact actions

© Rick Beerendonk

Appendix

What this lecture adds to the Microsoft Learn path

glasspaper oblicum

© Rick Beerendonk

What This Lecture Adds to the Microsoft Learn Path

This lecture supplements the official Microsoft Learn modules. It does not replace them.

Exam topic	In Microsoft Learn	Added in this lecture
Managed identity and keyless authentication	Appears in the authentication and agent lessons	Choice per scenario, how a token is issued, minimal roles
Private networking and role policies	Not developed as an architecture topic	Private endpoints, network rules, egress, data residency
Quotas, scaling, and cost footprints	Model cost, latency, and quota examples	RPM, TPM, and PTU, handling HTTP 429, capacity levers
CI/CD for Foundry projects	Only touched on briefly	Versioned assets, environments, evaluation before promotion
Monitoring, tracing, drift, and grounding quality	Evaluation and tracing are introduced, operations lightly	Signal per symptom, log categories, tracing settings, drift
Audit, provenance, approval, and oversight	Responsible AI and workflow approval, covered separately	Audit record, provenance, approval controls

Study beyond Microsoft Learn: authentication, cost, tracing, and approvals are only touched on there; private networking and the full operating model are exam topics that need this lecture or the product documentation.

glasspaper oblicum

© Rick Beerendonk